

SESIÓN 8

Observabilidad, Integración Total y Proyecto Final

Verificación de Integración Total y preparación de la Certificación en Vivo

Estudiante	Albert Omar Araya Cory
Repositorio	github.com/albrtaraya/fullstack-lab-1-1ci-cd
Prueba de fuego	PR #11 (frontend) y PR #17 (API)
Staging	fullstack-lab-1-1ci-cd-staging.up.railway.app
Modalidad	Individual
Fecha	22 de septiembre de 2026

Estado

Las 7 etapas verificadas · Prueba de fuego completa hasta staging
Guion de la demo y rúbrica de auditoría preparados

Resumen de ejecución

La Sesión 8 no trae conceptos nuevos. Su propósito es comprobar que las siete piezas construidas en las Sesiones 1 a 7 funcionan juntas como un sistema, y ensayar la demostración en vivo.

Sección	Contenido	Estado
1	Laboratorio final: Verificación de Integración Total	Cumplido
2	Preparación de la presentación en vivo	Preparado
3	Rol de auditor en la Certificación en Vivo	Preparado
4	Rúbrica de Certificación en Vivo	Autoevaluada

Lo más valioso del ensayo: verificar de verdad, y no solo revisar que las cosas existan, sacó a la luz un defecto real que ninguna sesión anterior había detectado. La imagen del backend construida desde main no arrancaba. Se detalla en la etapa 5.

1. Laboratorio final — Verificación de Integración Total

1.1 — El checklist de las 7 etapas

Se recorrió cada etapa en orden, sin avanzar hasta confirmar la anterior. Cada una se ejecutó en el equipo local y también en el pipeline.

Etapas	Qué hace, en una frase	Resultado
1. Lint	ESLint revisa el estilo y los errores comunes en cada push y PR	Sin hallazgos
2. Pruebas	Vitest ejecuta las pruebas unitarias, de componente y de API, con umbrales de cobertura	62/62, luego 65/65
3. E2E	Playwright recorre el flujo feliz en un navegador real	5/5
4. Build	Astro genera el sitio estático sin errores	1 página, 253 KB
5. Contenedores	Compose levanta PostgreSQL, API y frontend juntos	3 servicios sanos
6. Migraciones	Se aplican solas contra una base de prueba y son idempotentes	Verde en CI
7. Despliegue	Un merge a main despliega a staging, con rollback disponible	Staging en 2:27

Etapas 1 — Lint

```
$ npm run lint
> eslint .
exit=0
```

Etapas 2 — Pruebas unitarias, de componente y de API

La primera corrida local falló, pero no por el código:

```
Error: [vitest-pool]: Failed to start forks worker for test files ...
Caused by: Error: [vitest-pool-runner]: Timeout waiting for worker to respond
Test Files 4 passed (4) Tests 34 passed (34) Errors 3 errors
```

El equipo tenía 0,9 GB libres de 15,9 GB (94,6 % en uso), con Docker corriendo. Vitest no alcanzaba a levantar un proceso por archivo. Ejecutando los archivos de uno en uno pasan todas:

```
$ npx vitest run --no-file-parallelism
Test Files 7 passed (7)
Tests 60 passed (60)
```

Para la demo: si hay que correr las pruebas en vivo en este equipo, usar `--no-file-parallelism`. En el pipeline no hace falta, porque los runners tienen memoria de sobra.

Etapas 3 — Prueba E2E

Dos obstáculos locales, ambos del entorno y no de las pruebas:

Síntoma	Causa	Solución
Process from config.webServer exited	Quedaba un servidor de desarrollo en segundo plano, escuchando solo en IPv6 (:::1).	Levantar el servidor antes con <code>--host 127.0.0.1</code> para que

early	Playwright no lo detectaba e intentaba arrancar otro, que salía al instante	Playwright lo reutilice
2 de 4 pruebas fallaban esperando «0 tareas pendientes»	El stack de Compose seguía arriba: la app cargaba las 6 tareas reales de la API del puerto 4000	Apuntar el frontend a una API inexistente, igual que en el pipeline, donde no hay backend

```
$ PUBLIC_API_URL=http://127.0.0.1:4999 npx astro dev --background --host 127.0.0.1
$ npx playwright test
✓ un usuario puede crear una tarea y verla en la lista
✓ el contador de pendientes refleja las tareas agregadas
✓ no se agrega una tarea cuyo titulo son solo espacios
✓ el campo queda limpio despues de agregar una tarea
✓ la lista vacia invita a crear la primera tarea
5 passed (8.3s)
```

Etapas 4 — Build

```
$ npm run build
[build] 1 page(s) built in 8.44s
[build] Complete!
$ du -sh dist → 253K
```

Etapas 5 — Contenerización: el defecto que apareció

Tras un ciclo limpio de docker compose down y up, los tres servicios arrancaron. Pero /health respondía «Cannot GET /health»: el contenedor venía de una imagen del 15 de septiembre, anterior a la Sesión 7. Al reconstruir la imagen desde main, el backend entró en bucle de reinicio:

```
backend Restarting (1) Less than a second ago

at async aplicarMigraciones (file:///app/src/server/migraciones.js:52:23)
errno: -2, code: 'ENOENT', syscall: 'scandir',
path: '/app/db/migraciones'
```

Desde la Sesión 6 el servidor aplica las migraciones al arrancar, pero Dockerfile.backend solo copiaba src/. Nadie lo había notado porque el contenedor en ejecución era viejo. Ese mismo defecto habría roto el despliegue en Railway en cuanto construyera este Dockerfile. La corrección fue una línea:

```
COPY src ./src
# Las migraciones viajan dentro de la imagen: el servidor las aplica al
# arrancar, así que sin esta copia el contenedor muere buscando db/migraciones.
COPY db ./db

SERVICE      STATUS
backend       Up 21 seconds (healthy)
frontend      Up 2 minutes
postgres      Up 2 minutes (healthy)

$ curl localhost:4000/health → {"estado":"ok","status":"ok","almacen":"postgres"}
$ curl localhost:5173/      → 200
tareas persistidas tras down/up: 6
```

Etapla 6 — Migraciones y seed

En el pipeline, el job Migraciones y Seeds levanta un PostgreSQL efímero, aplica las migraciones y el seed, los repite para comprobar que son idempotentes y verifica que no se dupliquen filas. En staging, las migraciones corren al arrancar el servidor contra la base real.

Como parte del ensayo, Migraciones y Seeds se agregó como check requerido en la protección de main. Antes no lo era, y ahora sin ese check no se puede integrar:

```
"required_status_checks": ["Lint & Build", "Pruebas unitarias y cobertura",  
                           "Pruebas E2E", "Migraciones y Seeds"]  
"enforce_admins": true
```

Etapla 7 — Despliegue y rollback

Se cerró completa en la Sesión 7: despliegue automático a staging, despliegue manual a producción con la confirmación DEPLOY, y un rollback real practicado en el Simulacro de Caos. La prueba de fuego de la sección 1.2 lo vuelve a demostrar de punta a punta.

1.2 — La prueba de fuego: un cambio real de punta a punta

Se hizo en dos tiempos, porque el primer cambio elegido resultó ser del frontend y Railway publica solo la API.

Primer cambio: la lista vacía invita a crear la primera tarea

```
$ git checkout main && git pull origin main
$ git checkout -b feature/verificacion-final
$ git commit -m "fix: copiar las migraciones dentro de la imagen del backend"
$ git commit -m "feat: invitar a crear la primera tarea cuando la lista esta vacia"
$ git push origin feature/verificacion-final
```

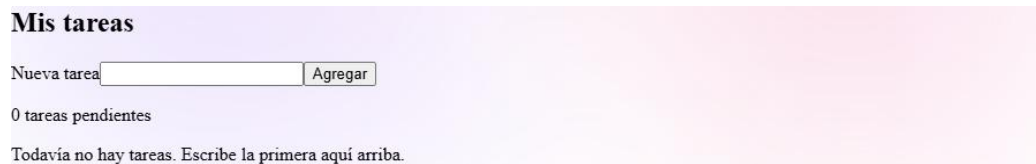


Figura 1. Antes de agregar tareas, la lista explica qué hacer en lugar de dejar un hueco.

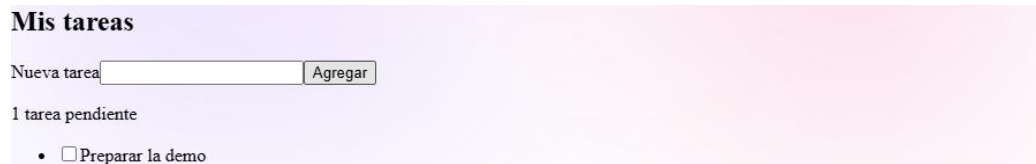


Figura 2. Con la primera tarea el mensaje desaparece y aparece la lista.

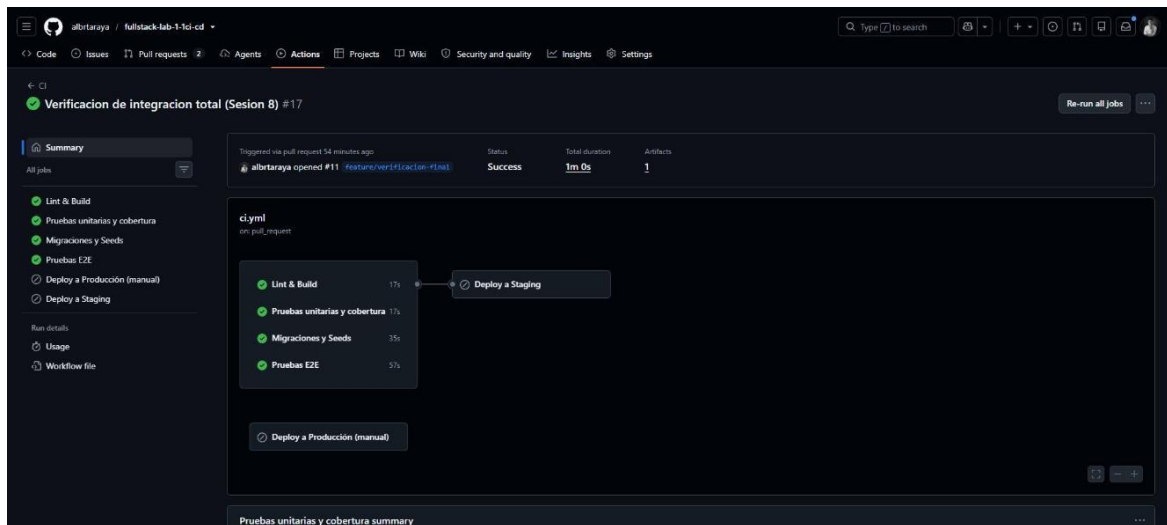


Figura 3. El Pull Request #11 dispara los cuatro controles en verde. Los despliegues se omiten porque es un PR.

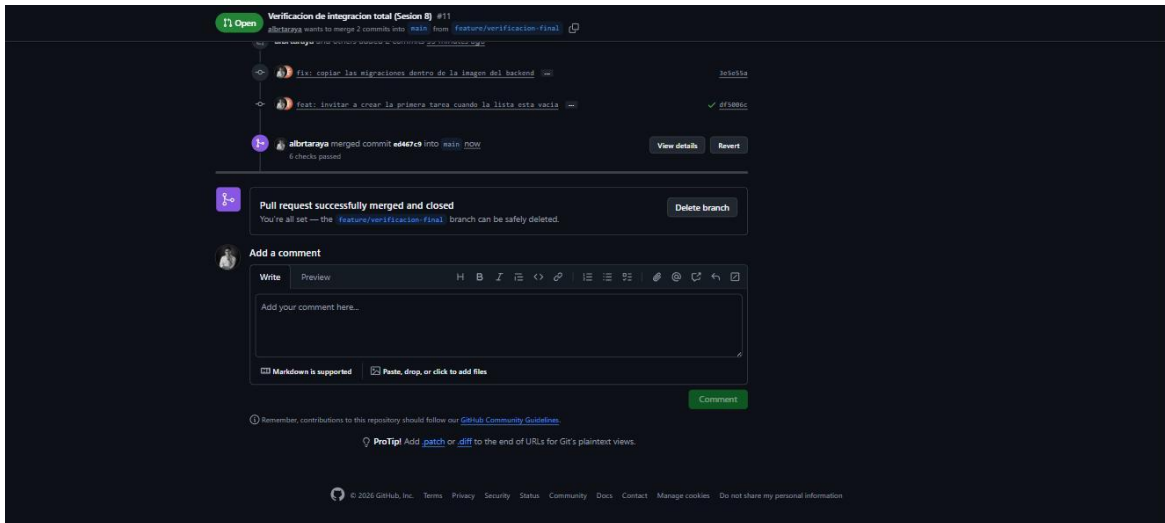


Figura 4. El Pull Request #11 integrado con los 6 checks.

El cambio quedó cubierto por una prueba de componente y una E2E nueva. Al integrarlo, el despliegue a staging todavía no se ejecutaba porque faltaba el token de Railway, que se completó en la Sesión 7. Además, Railway sirve solo la API, así que un cambio de pantalla no se ve en la URL de staging. Por eso se hizo un segundo cambio, del lado de la API.

Segundo cambio: resumen de tareas en la API

En una frase: la API ahora entrega un resumen de cuántas tareas hay pendientes y completadas, calculado con la misma función que usa el contador de la pantalla.

```
app.get('/tareas/resumen', async (req, res, next) => {
  try {
    const tareas = await repositorio.listar();
    const pendientes = contarTareasPendientes(tareas);
    res.status(200).json({
      total: tareas.length,
      pendientes,
      completadas: tareas.length - pendientes,
    });
  } catch (error) {
    next(error);
  }
});
```

Tres pruebas nuevas con Supertest y un repositorio de prueba: una lista con datos, la lista vacía y el almacén caído, que debe responder 500.

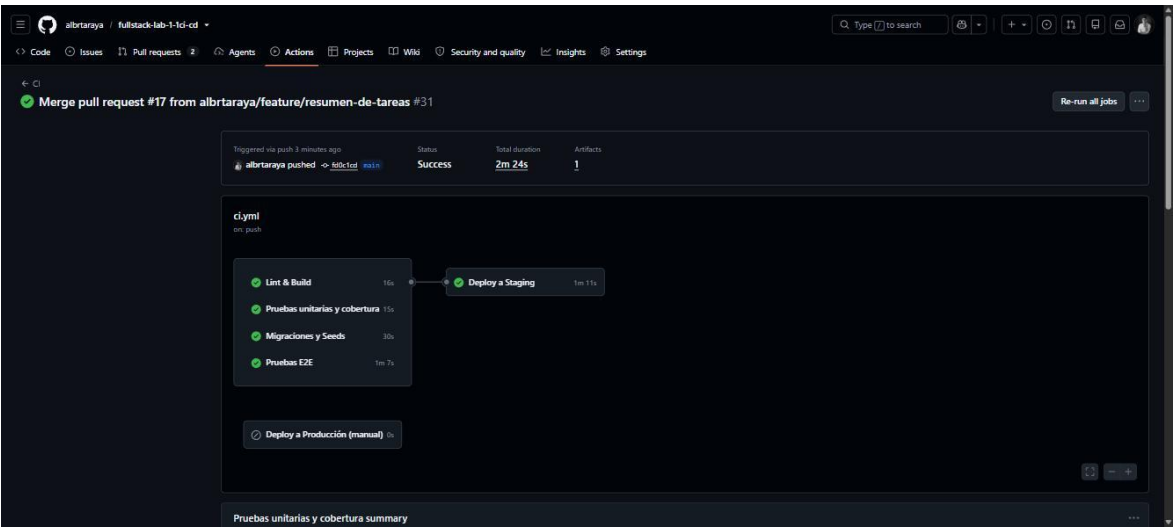


Figura 5. Tras el merge del PR #17: los cuatro controles y Deploy a Staging en verde (2 min 24 s). Producción, omitida.

Hora (UTC)	Evento
06:31:58	Merge del PR #17 a main; arranca el pipeline
06:32:09 – 06:34:15	Staging todavía sirve la versión anterior (404 en /tareas/resumen)
06:34:36	El endpoint responde en staging

Pretty-print ☐

```
{ "total": 1, "pendientes": 1, "completadas": 0 }
```

Figura 6. En el navegador: <https://fullstack-lab-1-1ci-cd-staging.up.railway.app/tareas/resumen>

Cannot GET /tareas/resumen

Figura 7. La misma ruta en producción responde 404: producción solo recibe cambios por despliegue manual.

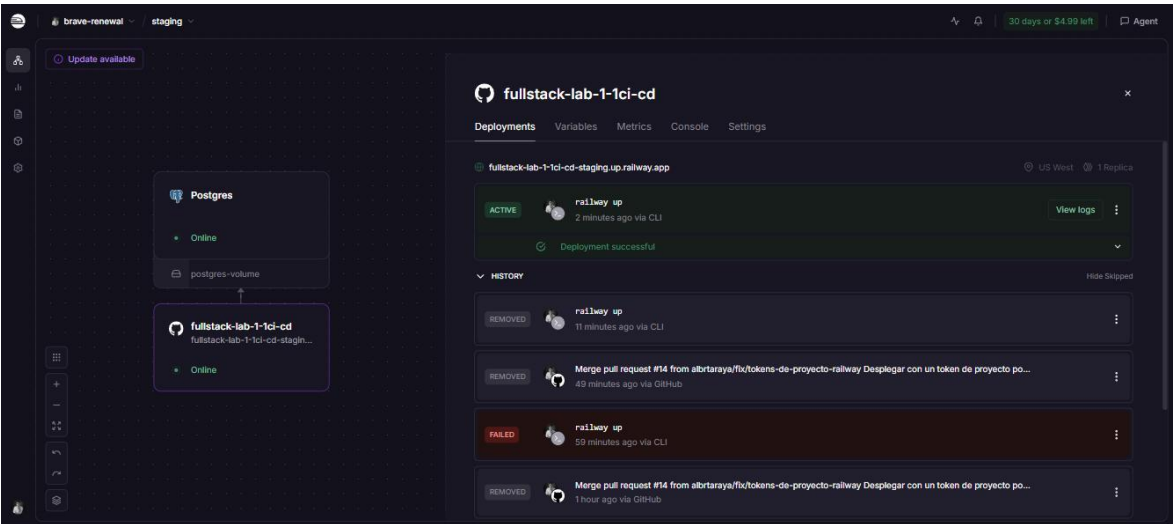


Figura 8. En Railway, el despliegue de la prueba de fuego queda ACTIVE sobre el historial del simulacro de la Sesión 7.

1.3 — Si algo falla: lo que realmente falló en el ensayo

Etapas	Qué falló	Dónde se revisó	Resolución
Pruebas	Timeout de los workers	Salida de Vitest y memoria del equipo	--no-file-parallelism
E2E	webServer exited early	Puertos con netstat y astro dev status	Servidor con --host 127.0.0.1
E2E	Contador distinto a 0	Puerto 4000 con curl /tarefas	Frontend sin API, como en CI
Docker	Bucle de reinicio	docker compose logs backend	COPY db ./db (PR #11)
Despliegue	Railway construía Astro	Log del job: «astro static site»	RAILWAY_DOCKERFILE_PATH

Checkpoint del Laboratorio Final

- ✓ Las 7 etapas del checklist están marcadas.
- ✓ Se ejecutó la prueba de fuego completa: un cambio real llegó de la rama hasta staging.
- ✓ Se sabe explicar qué hace cada etapa, no solo que existe (sección 2.3).

2. Preparación de la presentación en vivo

2.1 — Guion

Minuto	Qué mostrar
0 – 1	README y la frase: «una lista de tareas con API y base de datos, que se prueba, se contenedoriza y se despliega sola»
1 – 2	El cambio en vivo: agregar el porcentaje completado a /tareas/resumen, con su prueba
2 – 5	Push, Pull Request, los cuatro checks en verde, merge, Deploy a Staging y la URL de staging en el navegador
5 – 7	Preguntas del docente y de los compañeros

2.2 — Lo preparado con anticipación

1. Cambio de demo decidido: agregar porcentajeCompletado a /tareas/resumen. Es pequeño, se ve en el navegador sobre staging y se explica en una frase.
2. Tiempos medidos: el pipeline tarda unos 2 minutos y el cambio aparece en staging a los 2 min 27 s del merge. Se puede lanzar el push al empezar el minuto 1 para no esperar en silencio.
3. Pestañas abiertas de antemano: el PR, la pestaña Actions, Railway en staging y la URL /tareas/resumen.
4. Pruebas locales con --no-file-parallelism y E2E con el frontend sin API, por las condiciones del equipo descritas en la sección 1.1.

Las dos preguntas que hay que tener respondidas

«¿Qué pasaría si esta prueba fallara ahora?» El check de su job queda en rojo. Como main exige los cuatro checks, también a los administradores, el botón de merge no se habilita. Y aunque se pudiera integrar, Deploy a Staging depende de esos jobs con needs, así que nunca se ejecutaría.

«¿Cómo harías un rollback de este cambio?» Railway → staging → Deployments → menú : del último despliegue sano → Redeploy, como en el Simulacro de Caos. Después, un git revert por el flujo normal para que main vuelva a coincidir con lo desplegado. Antes de todo eso, el healthcheck ya impide que un despliegue que no arranca reemplace al que funciona.

2.3 — Cada etapa en una frase

1. Lint: ESLint rechaza el código con errores de estilo o variables sin usar antes de que alguien lo revise.
2. Pruebas: Vitest comprueba funciones, componentes y la API, y falla si la cobertura baja del umbral.
3. E2E: Playwright abre un navegador real y crea una tarea como lo haría un usuario.
4. Build: Astro genera el sitio; si no compila, no hay nada que desplegar.

5. Contenedores: Compose levanta base de datos, API y frontend idénticos en cualquier máquina.
6. Migraciones: el esquema se crea solo, en orden, y repetirlo no rompe nada.
7. Despliegue: un merge publica en staging, producción exige escribir DEPLOY y el healthcheck protege contra versiones que no arrancan.

2.4 — Si algo falla en vivo

La guía lo dice claro: un fallo real, diagnosticado con calma, demuestra más que una demo perfecta. Plan: leer el error en voz alta, ubicar la etapa y aplicar la tabla de la sección 1.3, que no es teórica, porque cada fila ya ocurrió de verdad durante el ensayo.

3. Certificación en Vivo — el rol de auditor

Durante las demos de los compañeros:

1. Marcar una etapa solo si se ve ejecutarse en pantalla, no si se menciona.
2. Si hay duda sobre si una etapa corrió en ese pipeline o en uno anterior, preguntarlo al final. Una forma rápida de saberlo es fijarse en la hora del run y en el commit.
3. Cerrar con una observación breve y genuina: algo bien resuelto o una pregunta técnica.

Qué mirar con ojo de auditor, por la experiencia propia: un Deploy a Staging en verde no prueba que se desplegó. En la Sesión 7 el job pasó en verde omitiendo el despliegue por falta de token. Hay que ver la URL respondiendo con el cambio nuevo.

4. Rúbrica — Autoevaluación del ensayo general

La certificación oficial la hacen los auditores durante la presentación en vivo. Esta tabla registra lo que se vio ejecutarse en el ensayo, con su evidencia:

Etapas del pipeline	¿Vista en ejecución?	Evidencia
1. Lint	Sí	Figura 5, job Lint & Build
2. Pruebas unitarias, de componente y de API	Sí	Figura 5 y sección 1.1
3. Prueba E2E	Sí	Figura 5, job Pruebas E2E
4. Build	Sí	Figura 5, job Lint & Build
5. Contenerización	Sí	Sección 1.1, etapa 5
6. Migraciones y seed automáticos	Sí	Figura 5, job Migraciones y Seeds
7. Despliegue automático a staging	Sí	Figuras 5, 6 y 8

Siete de siete en el ensayo, por encima del mínimo de cinco que exige la insignia  Graduado/a Misión DevOps (25 pts). La insignia se otorga en la Certificación en Vivo, cuando los auditores ven las etapas ejecutarse durante la presentación.

Criterio	Ponderación	Preparación
Claridad de la explicación técnica	30%	Frases por etapa (sección 2.3)
Manejo de preguntas e imprevistos	30%	Respuestas y tabla de fallos reales
Etapas certificadas en vivo	40%	7 de 7 verificadas en el ensayo

Anexo A — Índice de evidencias

Archivo	Contenido
01-demo-lista-vacia.png	La lista vacía con el mensaje nuevo
01b-demo-con-tarea.png	La lista con la primera tarea
02-run17-pr-checks.jpg	Run del PR #11: cuatro checks en verde
03-pr11-integrado.jpg	PR #11 integrado
04-run31-deploy-staging.jpg	Run #31: despliegue a staging tras el PR #17
05-staging-resumen.png	El endpoint nuevo en la URL de staging
06-produccion-sin-resumen.png	Producción sin el cambio: 404
07-railway-historial-staging.jpg	Historial de despliegues en Railway

Anexo B — Cambios de la sesión

PR	Commit	Qué cambió
#11	fix	Dockerfile.backend copia db/: la imagen de main ya arranca
#11	feat	Mensaje de lista vacía, con prueba de componente y E2E
#17	feat	GET /tareas/resumen, con tres pruebas de API
—	Ajuste	Migraciones y Seeds pasa a ser check requerido de main